MONITOR Z · L15.2

Clase 2 — La memoria y las direcciones

Cuaderno de estudio · versión del 2026-10-26

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L15 · Clase 2 — La memoria y las direcciones

En la Clase 1 viste que z/OS carga el programa «en un lugar libre de la memoria». Esta clase explica qué es ese lugar: cómo se numera la memoria, qué hay adentro de cada casillero y hasta dónde llega cada dirección.

> 📚 Fuentes: el listado del ensamblado de **JOB34775** (`monitor_z/salidas/JOB34775/…/ASM/SYSPRINT.txt`, sentencias 12631–12652 y 12178–12181, más el *Relocation Dictionary*), el mapa del binder del mismo job (paso `LKED`) y la corrida **JOB34879** del 22-10 (`+DIAG ANILLO` y el `IEF032I` de JESYSMSG). *z/OS MVS Programming: Assembler Services Guide*, capítulo «Using the 64-bit address space» (24, 31 y 64 bits).

1. La memoria es una fila de casilleros numerados

- Cada casillero guarda **un byte**, o sea 8 bits.
- El número del casillero es su **dirección**. Empieza en 0 y sube de a uno.
- Las direcciones se escriben en **hexadecimal**: cada dígito vale 4 bits, así que un byte son **2 dígitos exactos**.

Hexa	Decimal	Dónde lo viste
X'40'	64	el blanco en EBCDIC: `DC CL8' '` ensambla `4040404040404040`
X'1000'	4.096	donde empieza el piso 2 (R11)
X'1E2C'	7.724	donde está `NREP` dentro de ZTKMON
X'11170'	70.000	el largo del load module (mapa del binder)

2. Un byte no sabe qué es: depende de quién lo lea

Casos tomados de la zona de datos de ZTKMON (listado de JOB34775):

Fuente	Loc	Bytes	Cómo se lee
`MODO DC C'N'`	X'1E08'	`D5`	una letra: la N en EBCDIC

`MANIOK DC C'+DIAG …'`	X'22C7'	`4EC4C9C1C7…`	texto: `+` =4E `D` =C4 `I` =C9 `A` =C1 `G` =C7
`NREP DC F'0'`	X'1E2C'	`00000000`	un número binario de 4 bytes
`PUNTSSOB DC A(AREASSOB)`	X'1DDC'	`0000F4C8`	**una dirección guardada en memoria**: la de `AREASSOB`, que está en X'F4C8'

La CPU no distingue entre esos casos. Lo que decide es la instrucción: `CLI` lee un byte, `L` lee 4 bytes como número, y un `L` seguido de un uso como base trata esos 4 bytes como dirección.

La memoria: casilleros de un byte, numerados

L15 · Clase 2 · La memoria y las direcciones

Un pedazo real de ZTKMON (listado de JOB34775, desde X'1E08')

1E08	1E09	1E0A	1E0B	1E0C	1E0D	1E0E	1E0F	1E10	1E11	1E12	1E13	1E14	1E15	1E16	1E17
D5	**00**	**00**	**00**	**00**	**00**	**00**	**00**	**00**	**00**	**00**	**00**	**00**	**00**	**00**	**00**
MODO C'N'	relleno			**INTERV** F'0'				**DURMIN** F'0'				**ITER** F'0'			

Arriba: la **dirección** (el número del casillero, en hexa). Adentro: el **byte** (2 dígitos hexa = 8 bits).

Fuente	Bytes	Cómo se lee: lo decide la instrucción, no el byte
`MODO DC C'N'`	`D5`	una letra: la N en EBCDIC
`MANIOK DC C'+DIAG …'`	`4EC4C9C1C7`	texto: + D I A G
`NREP DC F'0'`	`00000000`	un número binario de 4 bytes
`PUNTSSOB DC A(AREASSOB)`	`0000F4C8`	**una dirección guardada en memoria**: la de AREASSOB

Analogía: los casilleros de un gimnasio

Cada casillero tiene un número pintado (la dirección) y guarda una sola cosa (el byte). Un papel adentro puede decir una palabra, un número… o «andá al casillero F4C8». El casillero no sabe cuál de las tres es.

Fuente: Listado ASM de JOB34775, sentencias 12631-12652 y 12817

Lámina L15-06_casilleros

3. Los tamaños y la alineación

Nombre	Bytes	En Assembler	Tiene que empezar en una dirección múltiplo de
byte	1	`C`, `X`	1
media palabra (*halfword*)	2	`H`	2
palabra (*fullword*)	4	`F`, `A`	4

doble palabra (*doubleword*)	8	`D`	8

HLASM alinea solo. Ejemplo real del listado:

Loc	Bytes	Fuente
X'1E08'	`D5`	`MODO DC C'N'` (1 byte)
X'1E09'	`000000`	**relleno**: 3 bytes que no escribiste
X'1E0C'	`00000000`	`INTERV DC F'0'`: X'1E0C' = 7.692 = 4 × 1.923

Lo mismo hace `TIEMPO DS 0D`, que queda en X'1DE0' (múltiplo de 8). El relleno ocupa lugar en las bases: cuando la Clase 6 mueva los datos a `ZTKCOM`, conviene agrupar los campos por tamaño para no desperdiciar bytes.

Los tamaños y la alineación

L15 · Clase 2 · La memoria y las direcciones

Nombre	Bytes	En Assembler	Empieza en múltiplo de
byte	1	`C` `X`	1
media palabra	2	`H`	2
palabra	4	`F` `A`	4
doble palabra	8	`D`	8

Lo que escribiste

```
MODO     DC    C'N'
INTERV   DC    F'0'
```

Dos líneas seguidas. Parece que INTERV va pegado a MODO.

Lo que armó HLASM

```
001E08 D5        MODO
001E09 000000    ← relleno
001E0C 00000000  INTERV
```

X'1E0C' = 7.692 = 4 × 1.923: **múltiplo de 4**.

HLASM alinea solo y el relleno ocupa lugar. También `TIEMPO DS 0D` quedó en X'1DE0', múltiplo de 8. Cuando los datos pasen a `ZTKCOM` (Clase 6), conviene agruparlos por tamaño para no perder bytes.

Fuente: Listado ASM de JOB34775, sentencias 12633 (TIEMPO) y 12638-12639 (MODO, INTERV)

Lámina L15-07_alineacion

4. La misma variable tiene dos direcciones

	Qué es	NREP
Dirección en el programa (*offset*, la columna `Loc` del listado)	cuántos bytes desde el comienzo de ZTKMON	X'1E2C', siempre la misma

Dirección real	dónde está el byte en la memoria mientras corre	dónde cargó z/OS + X'1E2C'. Cambia en cada corrida

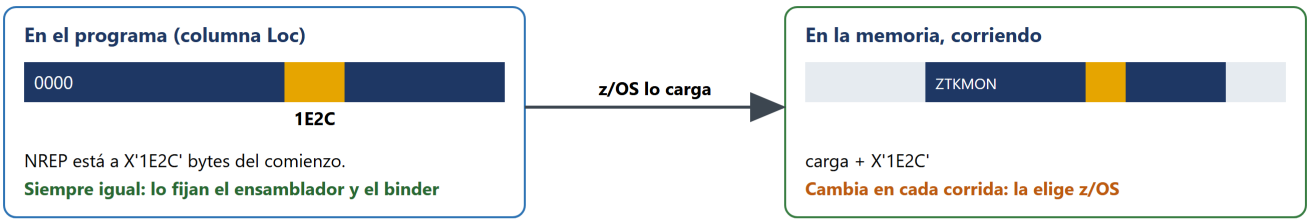
- El ensamblador y el binder sólo conocen la primera. La segunda existe recién cuando z/OS carga el programa.
- **Las constantes A se corrigen al cargar.** `PUNTSSOB` dice `0000F4C8` en el objeto, pero el ensamblador lo anotó en el

Relocation Dictionary (posición `00001DDC`, tipo `A`, 4 bytes, signo `+`). Al cargar, z/OS le **suma** la dirección donde puso el programa, y así queda la dirección real de `AREASSOB`.

- **No medido:** en qué dirección real quedó ZTKMON en JOB34879. El monitor no la informa.

La misma variable tiene dos direcciones

L15 · Clase 2 · La memoria y las direcciones



Caso: PUNTSSOB DC A(AREASSOB)	Qué pasa
En el objeto	`0000F4C8` : la dirección de AREASSOB **dentro del programa**
Relocation Dictionary	El ensamblador anotó «posición `00001DDC`, tipo A, 4 bytes, +»: a este campo hay que sumarle algo al cargar
Al cargar	z/OS le **suma** la dirección donde puso ZTKMON. Así queda la dirección real de AREASSOB

No medido: la dirección real donde quedó ZTKMON en JOB34879. El monitor no la informa.

Fuente: Listado ASM de JOB34775: sentencias 12631 y 12647, Relocation Dictionary (pág. 196)

Lámina L15-08_dos_direcciones

5. Hasta dónde llega una dirección: la línea y la barra

Bits de la dirección	Llega hasta	Nombre del límite
24	16 MB	**la línea** (*the line*)
31	2 GB	**la barra** (*the bar*)

64	16 exabytes	—

Así está ZTKMON hoy:

Qué	Dato medido	Dónde queda
El programa	`RMODE 24` (mapa del binder, JOB34775)	**debajo de la línea**
Las direcciones que puede usar	`AMODE 31` (mapa del binder)	cualquiera **debajo de la barra**
El anillo	`STORAGE OBTAIN,…,LOC=31` (sentencia 12181) y `+DIAG ANILLO CASILLAS=8192 BYTES=1114112 EN=19B00000` (JOB34879)	X'19B00000' = **411 MB**: arriba de la línea y debajo de la barra

La cuenta de JES lo confirma. El `IEF032I` de JOB34879 dice `VIRT: 92K` debajo de la línea y `EXT: 1124K` arriba:

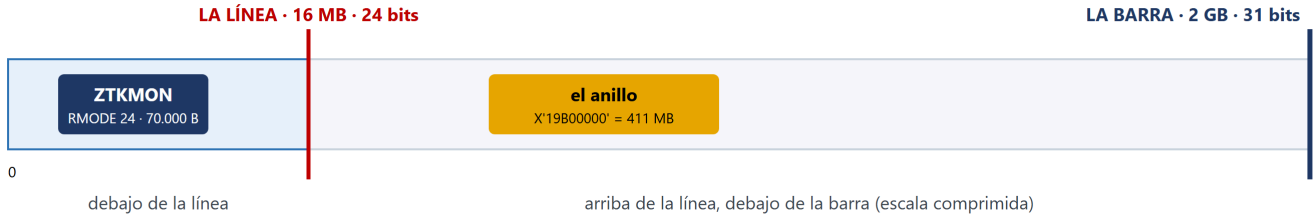
- debajo de la línea, 92 KB, entran los 70.000 bytes del programa (68,4 KB) y lo que pide el sistema;
- arriba de la línea, el anillo ocupa 1.114.112 bytes = 1.088 KB (8.192 casillas × 136 bytes) de los 1.124 KB. Los 36 KB

que sobran **no están atribuidos**: no se midió qué los ocupa.

Por qué el programa quedó en RMODE 24: el `DCB` de SYSPRINT (QSAM) tiene que estar debajo de la línea, y hoy el `DCB` está adentro del programa. Es el «cuidado especial» de la L15 (pregunta 7).

Hasta dónde llega una dirección: la línea y la barra

L15 · Clase 2 · La memoria y las direcciones



JES lo confirma (IEF032I de JOB34879):
`VIRT: 92K` debajo de la línea → el programa (68,4 KB) y lo del sistema · `EXT: 1124K` arriba → el anillo ocupa 1.088 KB

Qué	Dato medido	Dónde queda
El programa	`RMODE 24` (binder, JOB34775)	debajo de la línea, por el `DCB` de SYSPRINT
Sus direcciones	`AMODE 31` (binder, JOB34775)	puede usar cualquiera debajo de la barra
El anillo	`STORAGE OBTAIN,…,LOC=31` · `EN=19B00000`	arriba de la línea: 8.192 × 136 = 1.114.112 bytes

Sin atribuir: de los 1.124 KB de arriba, 36 KB no son del anillo, y no se midió qué los ocupa.

Fuente: Mapa del binder de JOB34775 · ASM sentencia 12181 · +DIAG ANILLO y IEF032I de JOB34879

Lámina L15-09_linea_y_barra

6. Qué cambia con los módulos

- **El área común `ZTKCOM` se va a pedir como el anillo**, con `STORAGE OBTAIN`. Puede ir arriba de la línea con `LOC=31`,

salvo el `DCB`, que necesita `LOC=24` o quedarse en el módulo principal.

- **Cada módulo va a llegar a los datos por una dirección guardada en un registro**, como `PUNTSSOB` guarda la de

`AREASSOB`, y no por su posición dentro del programa. Eso es lo que deja libres las bases.

7. Resumen

Idea	En una línea
Dirección	el número de un casillero de 1 byte
Un byte	no sabe qué es; la instrucción decide si es letra, número o dirección
Alineación	`F` y `A` empiezan en múltiplo de 4 y `D` en múltiplo de 8; HLASM rellena solo

Dos direcciones	la del listado (`Loc`) es fija; la real depende de dónde cargó z/OS
Línea y barra	16 MB (24 bits) y 2 GB (31 bits); ZTKMON está debajo de la línea y el anillo arriba

8. Preguntas de comprobación

1. En el listado, `MODO DC C'N'` está en X'1E08' e `INTERV DC F'0'` en X'1E0C'. ¿Qué hay en X'1E09'–X'1E0B' y por qué?
2. El anillo quedó en X'19B00000'. ¿Está arriba o abajo de la línea de 16 MB? ¿Qué parámetro del `STORAGE OBTAIN` lo permitió, y por qué el programa en sí no puede ir ahí todavía?

Mis respuestas

Notas